

Introduction To Algorithms

Week 4 - Search and Sort Behavior:
"Preconditions Make Algorithms Honest"

1

Terms to Remember

Term	Brief Meaning
Sorting algorithm	A procedure that arranges data into a defined order
Searching algorithm	A procedure that tries to locate a target value
Swap	Exchanging the values stored in two variables or positions
Linear search	Search that checks values one at a time
Binary search	Search that repeatedly cuts a sorted search space in half
Interpolation search	Search that estimates likely target position from value distribution
Precondition	Something that must be true before an algorithm is valid
Primary key	A value that uniquely identifies a record
Foreign key	A value that connects one record to another related record
NLP	Natural Language Processing; computing work involving human language

2

What We Will Use Today

- ▶ Sorting as preparation for efficient search
- ▶ Variable swapping
- ▶ Bubble, selection, insertion, merge, and shell sort recognition
- ▶ Linear search
- ▶ Binary search
- ▶ Sorted-data precondition
- ▶ Trace tables as evidence
- ▶ Primary/foreign key lookup as a brief application context

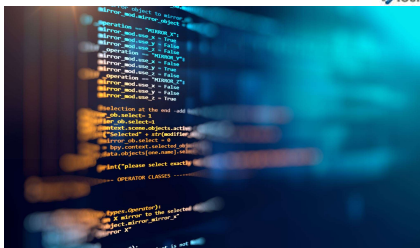
3

What We Will "Touch On" and Revisit Later

Deeper recursion in merge sort
Ranking and similarity
Hashing and lookup
Database indexing
Graph and tree traversal
Advanced NLP and data-modeling applications



4



Review Coding Lab

5

Review: What Lab 03 Taught Us

Last week, we focused on:

- How data was stored
- What operation mattered most
- Which structure made the operation clearer
- Which structure made the operation easier or faster

6



Today's Question

When is a faster algorithm not actually better?

7



What Counts as Success Today

- Explain the difference between sorting and searching
- Recognize several common sorting algorithms and their basic strategies
- Explain why algorithm choice depends on data size, order, and purpose
- Trace linear search and binary search at an introductory level
- Explain the sorted-data precondition for binary search
- Use evidence to show when an algorithm works, fails, or becomes unreliable

8



Preconditions Make Algorithms Honest

A precondition is something that must be true before the algorithm is valid.

Examples:

- ▶ binary search requires sorted data
- ▶ division requires a non-zero divisor
- ▶ a username lookup requires comparable username values
- ▶ a date sort requires consistent date representation

9



Searching And Sorting Work Together

Sorting asks:

- ▶ "How should the data be ordered?"

Searching asks:

- ▶ "How can we find the target?"

The order of the data can change the search strategy.

10



Textbook Review

Four Main Areas

The reading covers:

1. sorting algorithms
2. searching algorithms
3. performance analysis
4. practical applications

It also briefly defines Natural Language Processing.

11



Textbook Review

NLP Context

Natural Language Processing is computing work involving human language.

Examples:

- ▶ search suggestions
- ▶ document classification
- ▶ sentiment analysis
- ▶ text ranking
- ▶ chatbot response processing

This week is not an NLP lesson.

12

Textbook Review

Sorting

Sorting algorithms arrange values into a defined order.

The reading introduces:

- ▶ bubble sort
- ▶ selection sort
- ▶ insertion sort
- ▶ shell sort
- ▶ merge sort

13

Textbook Review

Searching

Searching algorithms try to find a target value.

The reading introduces:

- ▶ linear search
- ▶ binary search
- ▶ interpolation search

14

Textbook Review

Performance Is Interspersed

The reading discusses performance throughout the chapter.

For each algorithm, ask:

- ▶ What must be true before it works?
- ▶ How much work might it do?
- ▶ How does it behave as data grows?
- ▶ What tradeoff does it make?

15

Southwest Tech

Sorting Foundations

16

Southwest Tech

Variable Swapping

Many sorting algorithms need to swap two values.

Conceptually:

1. remember one value
2. move the other value
3. put the remembered value into the open spot

17

Southwest Tech

AFTER SWAP

Variable Swapping

18



Bubble Sort

Bubble sort repeatedly compares neighboring values.

If two neighbors are out of order:

- ▶ swap them
- ▶ continue across the list
- ▶ repeat passes until the list is sorted

19



Bubble Sort Performance

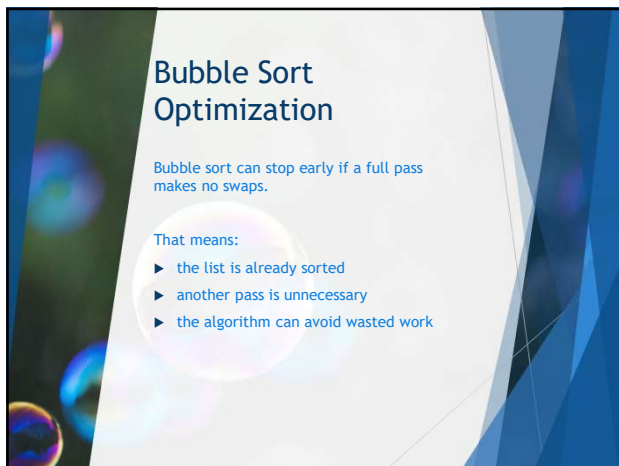
Bubble sort usually performs poorly on larger lists.

Why?

- ▶ it makes repeated passes
- ▶ it compares neighboring values
- ▶ it may keep checking even after little changes

Common growth label: $O(n^2)$

20



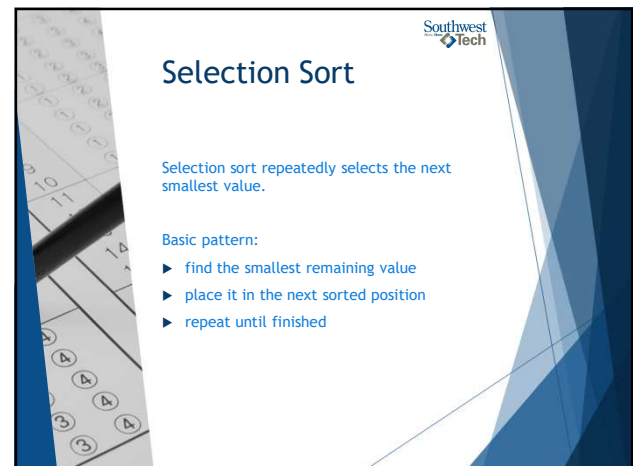
Bubble Sort Optimization

Bubble sort can stop early if a full pass makes no swaps.

That means:

- ▶ the list is already sorted
- ▶ another pass is unnecessary
- ▶ the algorithm can avoid wasted work

21



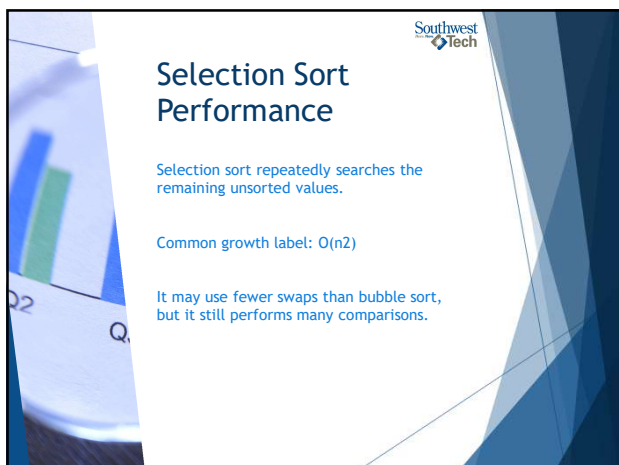
Selection Sort

Selection sort repeatedly selects the next smallest value.

Basic pattern:

- ▶ find the smallest remaining value
- ▶ place it in the next sorted position
- ▶ repeat until finished

22



Selection Sort Performance

Selection sort repeatedly searches the remaining unsorted values.

Common growth label: $O(n^2)$

It may use fewer swaps than bubble sort, but it still performs many comparisons.

23



Insertion Sort

Insertion sort builds a sorted section one value at a time.

Basic pattern:

- ▶ take the next value
- ▶ find where it belongs in the sorted section
- ▶ shift values if needed
- ▶ insert it

24



Insertion Sort Performance

Insertion sort depends heavily on existing order.

It can be:

- ▶ efficient for small lists
- ▶ efficient for nearly sorted data
- ▶ poor for badly unordered larger lists

Common worst-case growth label: $O(n^2)$

25

Shell Sort

Shell sort compares values that are farther apart first.

Then it gradually reduces the gap.

Purpose:

- ▶ move values closer to their final position sooner
- ▶ finish with smaller local adjustments

26



Merge Sort

Merge sort divides, sorts, and combines.

Basic pattern:

- ▶ split the data
- ▶ sort smaller pieces
- ▶ merge sorted pieces back together

Common growth label: $O(n \log n)$

27

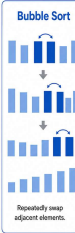


Sorting Strategy Comparison

Algorithm	Strategy	Taken Away
Bubble	compare neighbors	visible but inefficient
Selection	choose next smallest	organized repeated scanning
Insertion	insert into sorted section	useful for small or nearly sorted data
Shell	compare by gaps	refined insertion idea
Merge	divide and combine	stronger growth behavior

28





Bubble Sort

Repeatedly swap adjacent elements.



Selection Sort

Select the smallest and move it to the front.



Insertion Sort

Insert the current item into its correct position in the sorted left part.



Shell Sort

Compare and move elements that are far apart, then reduce the gap.



Merge Sort

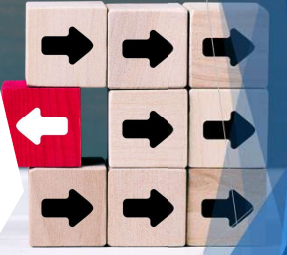
Divide into sorted groups and merge them together.

Sorting Comparison

29



Sorting Performance and Choice



30



Choosing A Sorting Algorithm

Choosing a sort depends on:

- ▶ data size
- ▶ current order
- ▶ memory limits
- ▶ need for simplicity
- ▶ need for predictable performance
- ▶ whether built-in sorting is acceptable

31



Simple Does Not Always Mean Wrong

A simple algorithm can be appropriate when:

- ▶ the data is small
- ▶ clarity matters most
- ▶ the cost is acceptable
- ▶ the goal is learning or verification

But simple should still be explained honestly.

32

Performance Claims: Needs Conditions



Avoid

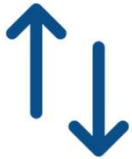
- ▶ "This is faster."
- ▶ "This is better."
- ▶ "This always works."



Use

- ▶ "This is faster when..."
- ▶ "This is valid if..."
- ▶ "This performs better for..."

33

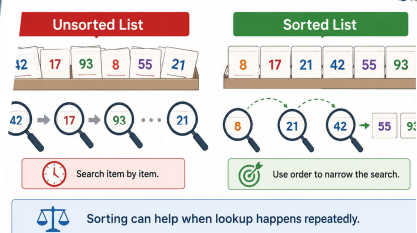


Sorting As Preparation

Sorting may be useful before searching when:

- ▶ many searches will happen
- ▶ the data will be reused
- ▶ the search strategy depends on order
- ▶ the sorting cost is worth the later benefit

34



Sorting As Preparation

Sorting can help when lookup happens repeatedly.

35



From Sort To Search

Sorting changes what search can assume.

Unsorted data:

- ▶ Linear search is safe
- ▶ Binary search is not trustworthy

Sorted data:

- ▶ Linear search still works
- ▶ Binary search can use the order

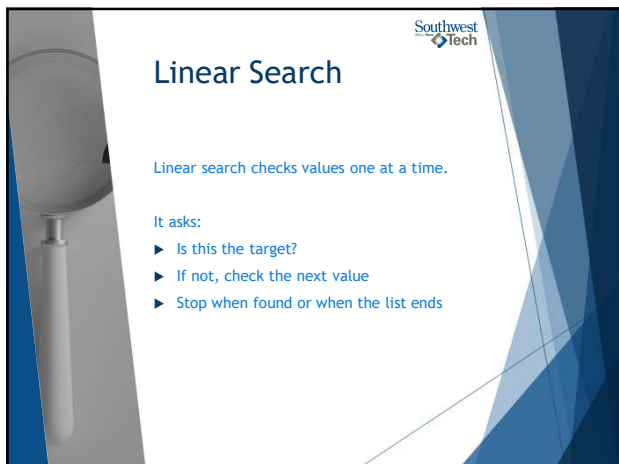
36



37



38



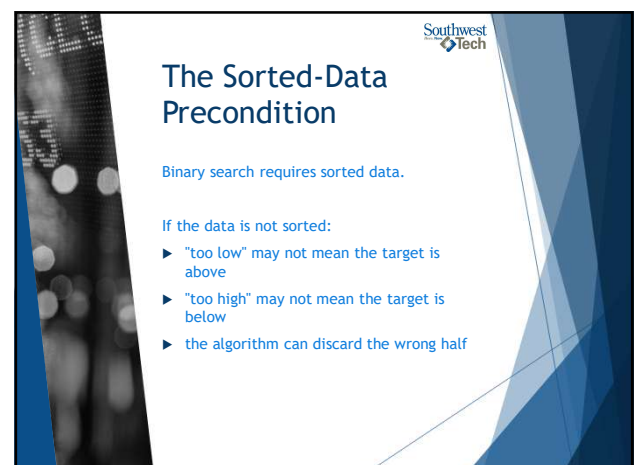
39



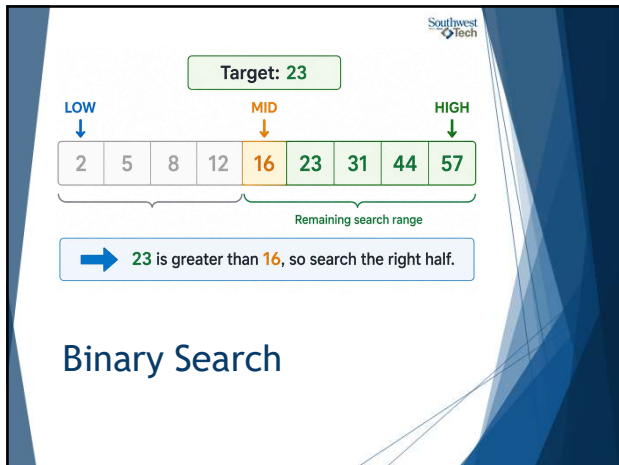
40



41



42



43

Step	Low	High	Mid Pos	Value	Decision
1	2	57	5th	16	Search greater half
2	16	57	7th (3 rd)	31	Search lower half
3	16	31	8th (2 nd)	23	Value found - STOP

Binary Search Trace (Table)

Target: 23

44

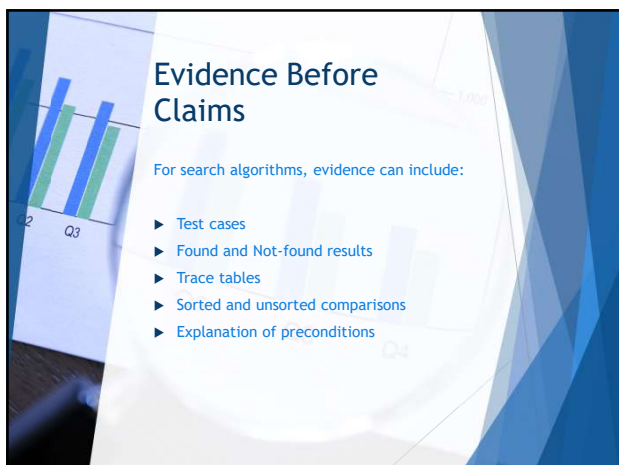


45

Search Strategy Comparison

Search	Needs Sorted Data?	Basic Idea	Common Growth
Linear	NO	Check one at a time	$O(n)$
Binary	YES	Cut search space in half	$O(\log n)$
Interpolation	YES, plus distribution assumptions	Estimate likely position	varies

46



47



48

Practical Application Context

49

Records And Keys

Real systems often search records by keys.

Examples:

- ▶ product ID
- ▶ student ID
- ▶ ticket number
- ▶ course code
- ▶ username

50

Primary And Foreign Keys

Primary key:

- ▶ uniquely identifies one record

Foreign key:

- ▶ connects a record to another related record

Search and sort help systems find and connect records.

51

Students

StudentID	StudentName
101	Maya
102	Jordan
103	Luis

Primary Key

Primary Key (unique identifier)

Lab Scores

ScoreID	StudentID	LabScore
1	101	92
2	102	85
3	101	96

Foreign Key

Foreign Key (references primary key)

Primary and Foreign Keys

52

Historical Database Example

The reading's historical database example asks:

- ▶ What identifies a record?
- ▶ What connects records?
- ▶ What should be searched?
- ▶ What should be sorted?
- ▶ What lookup pattern matters most?

53

NLP Connection, Briefly

NLP (Natural Language Processing) systems may use search and sort ideas to:

- ▶ find matching terms
- ▶ rank documents
- ▶ sort likely responses
- ▶ retrieve related records
- ▶ compare text features

This is context, not today's coding target.

54

Demo Scenario

Problem: Searching book titles on a shelf

Goal

Binary Search results on: Sorted vs Unsorted data

55

Demo Evidence

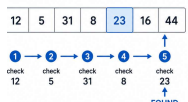
The demo produces:

- ▶ a linear search trace
- ▶ a binary search trace on sorted data
- ▶ a binary search trace on unsorted data
- ▶ a sorted-vs-unsorted summary

56

Linear Search Trace

Target: 23



Binary Search Trace

Target: 23



Trace evidence shows how the algorithm reached the answer.

Demo Evidence

57

Demo Key Question

Ask after each search:

- ▶ What did the algorithm check?
- ▶ What did it assume?
- ▶ What did it ignore?
- ▶ Was that decision valid?

58

Demo Explanation

Example explanation:

"Binary search is valid on the sorted list because each comparison tells us which half can be ignored. On the unsorted list, that decision is unreliable because the values are not arranged in order."

59

From Demo to Lab

In Lab 04, you will use a different data set.

You must show:

- ▶ linear search
- ▶ binary search
- ▶ found and not-found tests
- ▶ binary search attempted on unsorted data
- ▶ trace evidence
- ▶ precondition explanation

60

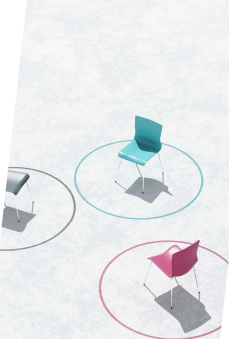


Walkthrough Support

The walkthrough artifact can help you:

- ▶ ?

If you use it, you must explain the pattern in your own words, what you changes, and your decisions:



61



README Evidence



Your README should show:

- ▶ test case table
- ▶ trace table
- ▶ sorted-data precondition explanation
- ▶ comparison of linear and binary search
- ▶ AI-use note, if used note, if applicable

62



AI Use In Lab 04



Manual first:

- ▶ Write or trace your own attempt
- ▶ Create your test cases
- ▶ Identify the precondition

AI-assisted after:

- ▶ Review logic
- ▶ Check edge cases
- ▶ Explain a confusing trace
- ▶ Suggest tests

63



Wrap up



64



Lab On Success Check



- ▶ Searches correctly
- ▶ Includes required test cases
- ▶ Traces at least one search
- ▶ Explains sorted-data requirements
- ▶ Avoids claiming binary search is always better

65



What To Carry Forward



Search and sort algorithms teach a larger lesson:

- ▶ Algorithms depend on data conditions
- ▶ Performance depends on context
- ▶ Correctness includes assumptions
- ▶ Evidence makes reasoning visible

66

Southwest
Tech

Next Step

1

For Lab 04:

- choose a data set
- implement or simulate linear search
- implement or simulate binary search
- create required tests
- build a trace table
- explain the sorted-data precondition

67

Southwest
Tech

How To Use The Textbook For Next Week's Reading



For next week, read for strategy choice.

Focus on:

- ▶ Correctness, performance, and scalability
- ▶ Functional and non-functional requirements
- ▶ Divide and conquer
- ▶ Dynamic programming
- ▶ Greedy strategies
- ▶ PageRank and linear programming as examples

Skim the P, NP-Hard, and NP-Complete section for recognition only.

68

Southwest
Tech



Thank you!

Have Fun!

69